MON APPRENTISSAGE — SERIES PYTHON

Python

Du zéro à l'expert

Chapitre 12 — Les Fonctions de Base

Organiser, réutiliser, abstraire — la puissance des fonctions

Auteur :

KETOTSA AMÉVI CLAUDE

Date de publication :

10 mars 2026

Publié dans le cadre de mon parcours d'apprentissage Python

Table des matières

Chapitre 12 — Les Fonctions de Base **2**
0.1 Définir et appeler une fonction 2
0.2 Les paramètres et arguments 2
0.3 La valeur de retour — return 3
0.4 Les valeurs par défaut 4
0.5 La portée des variables — Scope 5
0.6 Retourner plusieurs valeurs 7
0.7 Les fonctions comme objets 7
0.8 Documenter ses fonctions 8
0.9 Ce qui m'a le plus surpris dans ce chapitre 9
0.10 Résumé du Chapitre 12 10

Chapitre 12 — Les Fonctions de Base

À retenir

Une fonction est un **bloc de code réutilisable** auquel on donne un nom. Au lieu de réécrire les mêmes instructions dix fois, on les encapsule dans une fonction et on l'appelle quand on en a besoin. C'est le premier grand pas vers un code **organisé, lisible et maintenable**. Les fonctions sont le cœur de tout programme sérieux.

Définir et appeler une fonction

En Python, une fonction se définit avec le mot-clé `def`.

Définir et appeler une fonction

```python
# Syntaxe de base
def saluer():
    print("Bonjour tout le monde !")
    print("Bienvenue en Python.")

# Appeler la fonction
saluer()    #     Bonjour tout le monde !
saluer()    #     Bienvenue en Python.
saluer()    # R utilisable    l'infini !

# Fonction avec un peu plus de logique
def afficher_separateur():
    print("=" * 40)

def afficher_menu():
    afficher_separateur()
    print("      MENU PRINCIPAL")
    afficher_separateur()
    print("  1. Nouvelle partie")
    print("  2. Charger")
    print("  3. Quitter")
    afficher_separateur()

afficher_menu()
```

Les paramètres et arguments

Les **paramètres** sont les variables déclarées dans la définition de la fonction. Les **arguments** sont les valeurs passées lors de l'appel.

Paramètres et arguments

```python
# Fonction avec un parametre
def saluer(prenom):
    print(f"Bonjour, {prenom} !")

saluer("Alice")     #     Bonjour, Alice !
saluer("Bob")       #     Bonjour, Bob !
saluer("Charlie")   #     Bonjour, Charlie !

# Fonction avec plusieurs parametres
def presenter(prenom, age, ville):
    print(f"Je m'appelle {prenom}.")
    print(f"J'ai {age} ans et j'habite    {ville}.")

# Arguments positionnels     l'ordre compte
presenter("Alice", 25, "Paris")

# Arguments nommes (keyword arguments)     l'ordre ne compte
    pas
presenter(age=30, ville="Lyon", prenom="Bob")

# M langer positionnels et nommes
presenter("Charlie", ville="Marseille", age=22)
```

Erreur fréquente

Les arguments **positionnels** doivent toujours précéder les arguments **nommés** lors d'un appel de fonction. Écrire `presenter(prenom="Alice", 25, "Paris")` lève une `SyntaxError`. L'ordre est : positionnels d'abord, nommés ensuite.

La valeur de retour — return

`return` permet à une fonction de **renvoyer un résultat** à celui qui l'a appelée.

return — retourner une valeur

```python
# Sans return     la fonction retourne None implicitement
def afficher_double(n):
    print(n * 2)

resultat = afficher_double(5)    # Affiche 10
print(resultat)                  # None !

# Avec return     la fonction retourne une valeur utilisable
def calculer_double(n):
    return n * 2

resultat = calculer_double(5)
```

```python
print(resultat)           # 10
print(resultat + 3)       # 13
print(calculer_double(7) * 2)    # 28

# Fonctions mathematiques
def aire_rectangle(longueur, largeur):
    return longueur * largeur

def perimetre_rectangle(longueur, largeur):
    return 2 * (longueur + largeur)

l, L = 5, 3
print(f"Aire : {aire_rectangle(l, L)} cm ")
print(f"Perimetre : {perimetre_rectangle(l, L)} cm")

# return arrete l'execution de la fonction
def verifier_age(age):
    if age < 0:
        return "Age invalide"
    if age < 18:
        return "Mineur"
    return "Majeur"

print(verifier_age(-5))    # Age invalide
print(verifier_age(15))    # Mineur
print(verifier_age(25))    # Majeur
```

À retenir

Une fonction sans `return` retourne `None` implicitement. `return` peut apparaître **plusieurs fois** dans une fonction — dès qu'il est atteint, la fonction s'arrête et renvoie la valeur. C'est un excellent moyen de gérer plusieurs cas sans `elif` imbriqués.

Les valeurs par défaut

On peut définir des valeurs par défaut pour les paramètres — rendant certains arguments optionnels.

Paramètres avec valeurs par défaut

```python
# Valeur par defaut avec =
def saluer(prenom, message="Bonjour"):
    print(f"{message}, {prenom} !")

saluer("Alice")                    # Bonjour, Alice !
saluer("Bob", "Bonsoir")           # Bonsoir, Bob !
saluer("Charlie", "Salut")         # Salut, Charlie !
```

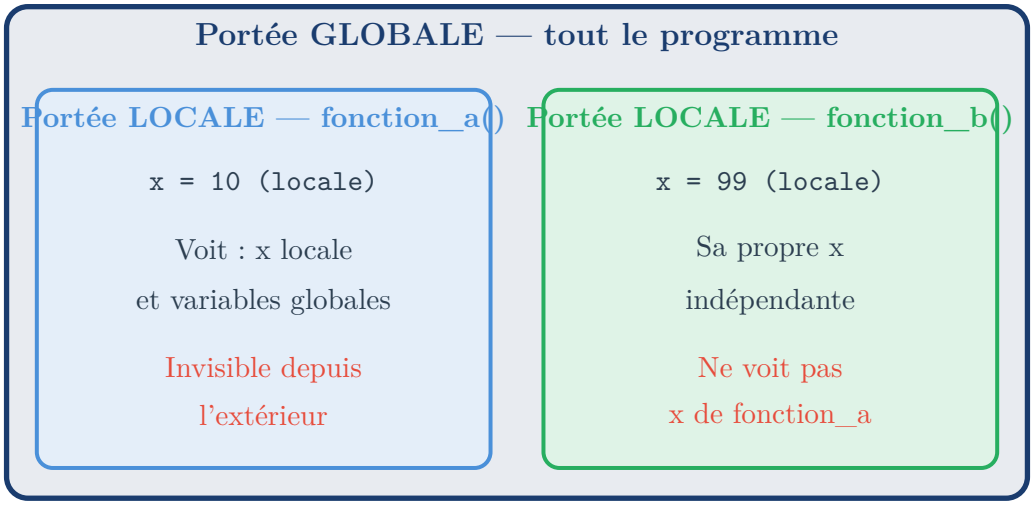
```python
# Plusieurs valeurs par defaut
def creer_profil(prenom, age=0, ville="Inconnue", actif=True):
    return {
        "prenom": prenom,
        "age":    age,
        "ville":  ville,
        "actif":  actif
    }

p1 = creer_profil("Alice", 25, "Paris")
p2 = creer_profil("Bob")
p3 = creer_profil("Charlie", ville="Lyon", age=30)

print(p1)
print(p2)    # age=0, ville="Inconnue", actif=True
print(p3)
```

Erreur fréquente

Les paramètres avec valeur par défaut doivent toujours être placés **après** les paramètres sans défaut. `def f(age=0, prenom)` est une `SyntaxError`. La règle : obligatoires d'abord, optionnels ensuite.

La portée des variables — Scope

La **portée** (scope) définit où une variable est accessible dans le programme.



Portée locale et globale

```python
# Variable GLOBALE
compteur = 0
message = "Bonjour"

def afficher():
```

```python
    # Lecture de la variable globale     OK
    print(message)    # Bonjour

def incrementer_faux():
    #     Cree une variable LOCALE, pas globale
    compteur = 10
    print(compteur)    # 10 (local)

def incrementer_vrai():
    #     global d clare qu'on veut modifier la globale
    global compteur
    compteur += 1
    print(compteur)

afficher()
incrementer_faux()
print(compteur)       # 0     inchange !
incrementer_vrai()
print(compteur)       # 1     modifie !
incrementer_vrai()
print(compteur)       # 2
```

nonlocal — fonctions imbriquées

```python
def externe():
    x = 10

    def interne():
        nonlocal x    # modifie x de la fonction externe
        x += 5
        print(f"Interne : x = {x}")

    interne()
    print(f"Externe : x = {x}")

externe()
#     Interne : x = 15
#     Externe : x = 15
```

Ce qui m'a résisté

Le scope m'a demandé beaucoup de temps à comprendre. J'avais instinctivement supposé qu'une variable modifiée dans une fonction affectait la variable globale du même nom. C'est faux — Python crée une **variable locale** distincte. Le mot-clé `global` est nécessaire pour modifier une variable globale depuis une fonction. En pratique, on évite `global` autant que possible — il vaut mieux **retourner** une valeur et réassigner.

Retourner plusieurs valeurs

Une fonction Python peut retourner **plusieurs valeurs** en meme temps via un tuple.

Retourner plusieurs valeurs

```python
def diviser(a, b):
    if b == 0:
        return None, "Division par zero impossible"
    quotient  = a // b
    reste     = a % b
    return quotient, reste   # retourne un tuple implicite

# Deballage direct
q, r = diviser(17, 5)
print(f"17    5 = {q} reste {r}")   # 17    5 = 3 reste 2

# Statistiques completes
def statistiques(liste):
    n       = len(liste)
    total   = sum(liste)
    moyenne = total / n
    minimum = min(liste)
    maximum = max(liste)
    return n, total, moyenne, minimum, maximum

notes = [15, 8, 18, 12, 17, 9]
n, total, moy, mini, maxi = statistiques(notes)
print(f"N={n} | Total={total} | Moy={moy:.1f}")
print(f"Min={mini} | Max={maxi}")
```

Les fonctions comme objets

En Python, les fonctions sont des **objets de première classe** — on peut les assigner à des variables, les passer en argument, les stocker dans des listes.

Les fonctions sont des objets

```python
def doubler(x):
    return x * 2

def tripler(x):
    return x * 3

# Assigner une fonction a une variable
operation = doubler
print(operation(5))   # 10

operation = tripler
print(operation(5))   # 15
```

```python

# Passer une fonction en argument
def appliquer(fonction, valeur):
    return fonction(valeur)

print(appliquer(doubler, 7))    # 14
print(appliquer(tripler, 7))    # 21

# Stocker des fonctions dans une liste
operations = [doubler, tripler]
for op in operations:
    print(op(10))    # 20, puis 30

# Stocker dans un dictionnaire
calculs = {
    "double": doubler,
    "triple": tripler
}
print(calculs["double"](6))    # 12
```

À retenir

Le fait que les fonctions soient des **objets de première classe** en Python est fondamental. Cela ouvre la voie aux décorateurs, aux callbacks, à la programmation fonctionnelle — tous les concepts avancés que nous verrons dans les chapitres suivants. Une fonction n'est pas juste un bloc de code — c'est un objet comme les autres.

Documenter ses fonctions

Une bonne fonction est une fonction **documentée**. Les docstrings permettent d'expliquer ce que fait une fonction.

Docstrings — documenter ses fonctions

```python
def calculer_imc(poids, taille):
    """
    Calcule l'Indice de Masse Corporelle (IMC).

    Parametres :
        poids (float) : le poids en kilogrammes
        taille (float) : la taille en metres

    Retourne :
        float : l'IMC arrondi a 2 d cimales
        str   : la categorie correspondante

    Exemple :
        >>> calculer_imc(70, 1.75)
```

```
        (22.86, 'Poids normal')
    """
    imc = poids / (taille ** 2)
    imc = round(imc, 2)

    if imc < 18.5:
        categorie = "Insuffisance ponderale"
    elif imc < 25:
        categorie = "Poids normal"
    elif imc < 30:
        categorie = "Surpoids"
    else:
        categorie = "Obesite"

    return imc, categorie

# Acceder a la documentation
print(calculer_imc.__doc__)

# Utilisation
imc, cat = calculer_imc(70, 1.75)
print(f"IMC : {imc}     {cat}")
```

Ce qui m'a le plus surpris dans ce chapitre

Ce qui m'a résisté

Trois révélations de ce chapitre :

1. Une fonction sans `return` retourne `None`. J'avais écrit `resultat = ma_fonction()` en pensant récupérer le résultat affiché par `print()` à l'intérieur. En réalité, `print()` affiche mais ne retourne rien. `return` est obligatoire pour qu'une valeur sorte de la fonction. Cette distinction entre **afficher** et **retourner** est fondamentale et j'aurais aimé la comprendre plus tôt.

2. Le scope local crée une variable indépendante. Modifier une variable à l'intérieur d'une fonction ne modifie **pas** la variable globale de même nom. Python crée une variable locale distincte. C'est une protection contre les effets de bord — mais ça m'a causé des bugs incompréhensibles avant de comprendre la règle LEGB (Local, Enclosing, Global, Built-in) qui définit l'ordre de résolution des noms en Python.

3. Les fonctions sont des objets — pas juste des instructions. Découvrir qu'on peut stocker une fonction dans une variable, la passer en argument à une autre fonction, la mettre dans une liste — c'est un changement de paradigme. En Python, une fonction n'est pas une procédure figée dans le code. C'est un **objet vivant** qu'on peut manipuler comme n'importe quelle autre valeur. Cette réalisation ouvre la porte à des possibilités infinies.

Résumé du Chapitre 12

Ce que j'ai appris dans ce chapitre :

- Définir une fonction avec `def nom(params):` et l'appeler avec `nom(args)`.
- **Paramètres positionnels** et **nommés** — l'ordre et la flexibilité.
- `return` renvoie une valeur — sans lui, la fonction retourne `None`.
- **Valeurs par défaut** — rendre certains arguments optionnels.
- **Scope** : local, global, `global`, `nonlocal` — chaque variable a sa portée.
- Retourner **plusieurs valeurs** via des tuples implicites.
- Les fonctions sont des **objets** — assignables, passables, stockables.
- **Docstrings** — documenter ses fonctions pour soi et pour les autres.

*"Une fonction bien écrite fait **une chose**, la fait **bien**, et la fait **une seule fois**."*